Robotics and Flexible Automation

Lecture 4
Inverse kinematics

Prof. Zengxi (Stephen) Pan

BJTU · 2026

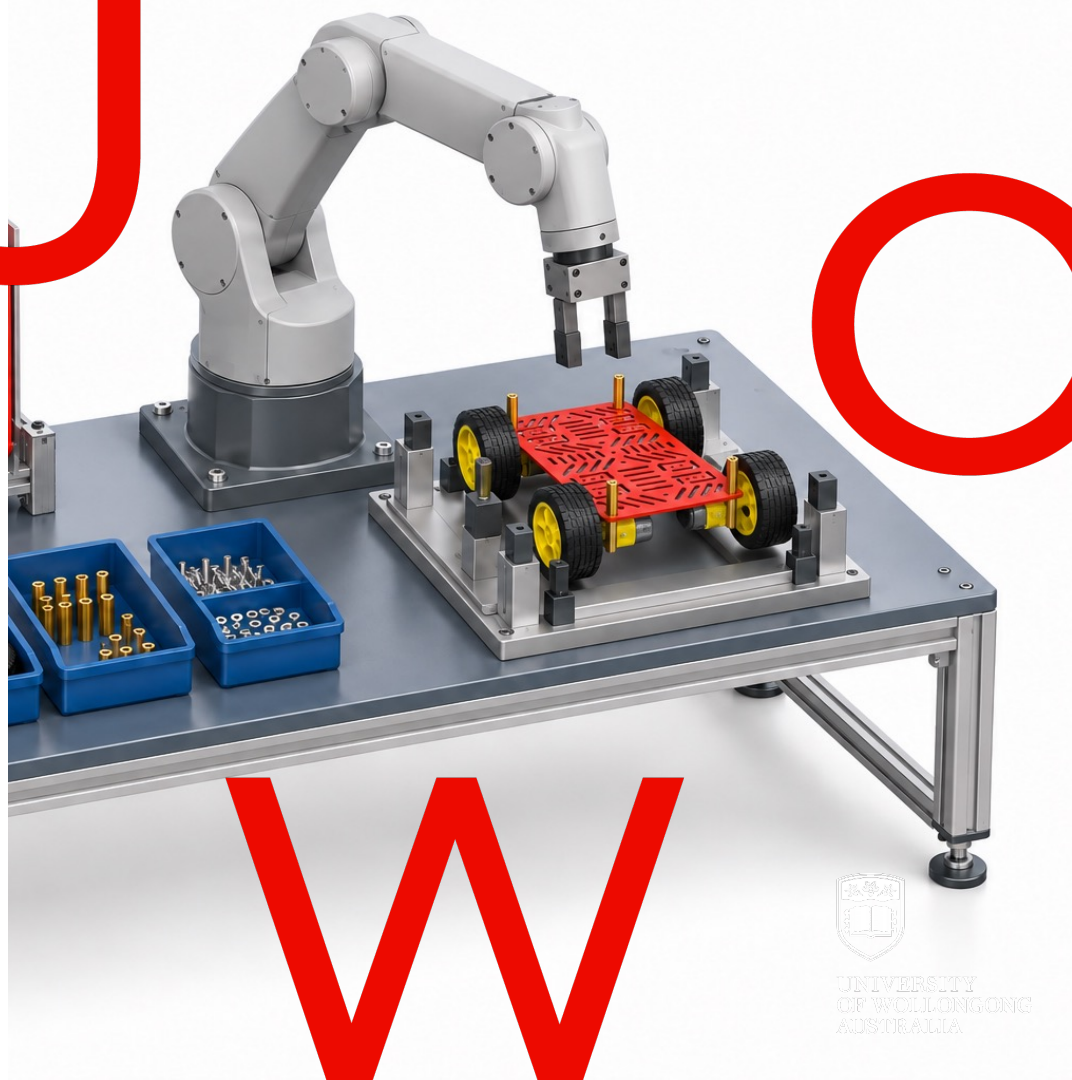


Table of Contents

CORE

1 The IK problem and configurations

2 Analytic branches and reachability

3 Numerical IK and singularities

4 Spherical wrist and toolbox IK

PART 1

The IK problem and configurations

A target pose can map to more than one joint solution

1. Forward versus inverse kinematics
2. Target pose, configuration and reachability
3. The inverse-kinematics challenge

Forward and Inverse Kinematics

CORE

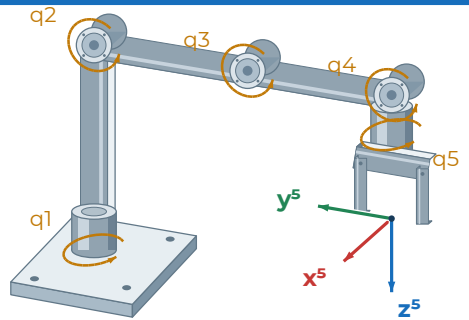
This class: find joint coordinates q for a desired tool pose T

Forward kinematics (last class)

Joint coordinates

$$q = [q_1 \ \dots \ q_n]^T$$

Joint angles / displacements



Tool pose:
Cartesian coordinates

T

Position and orientation

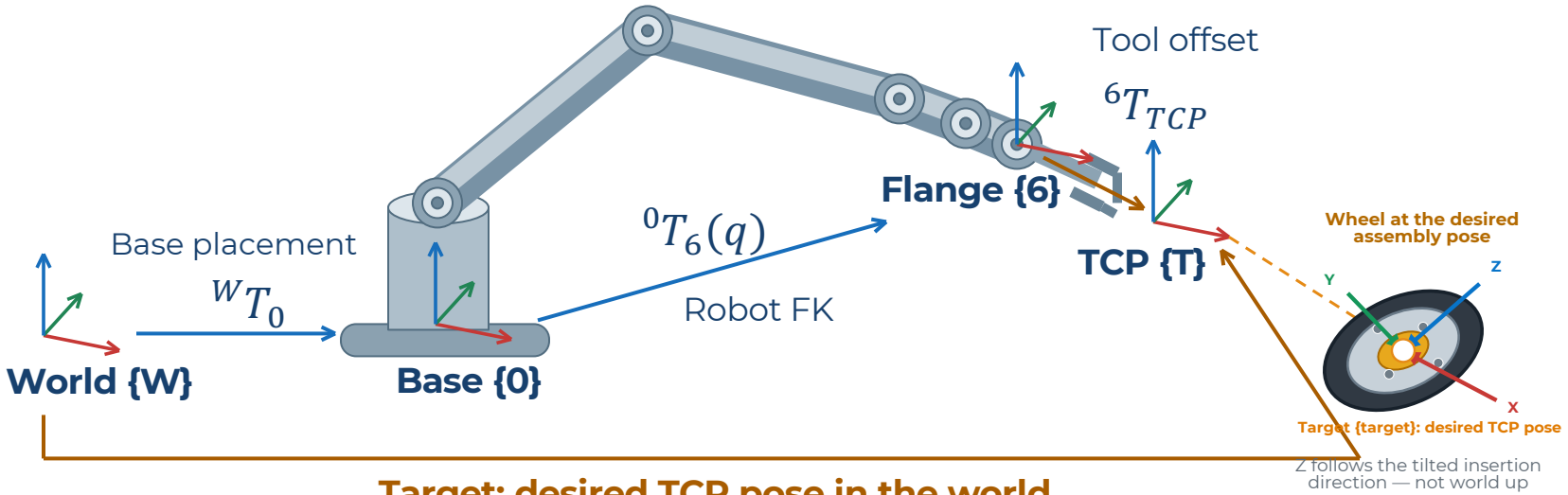
Inverse kinematics (this class)

Forward: a joint configuration determines one tool pose.
Inverse: a desired pose may have no solution, one solution or multiple solutions.

The Target Pose

CORE

Frame conversion: world TCP target + base placement + tooldata give the flange target.



Target: desired TCP pose in the world

$$ {}^{W}T_{TCP}(q) = {}^{W}T_{0}\,{}^{0}T_{6}(q)\,{}^{6}T_{TCP} = {}^{W}T_{target} $$

$$ {}^{0}T_{6}(q) = {}^{W}T_{0}{}^{-1}\,{}^{W}T_{target}\,{}^{6}T_{TCP}{}^{-1} $$

Controller IK: ${}^{0}T_{6}(q)$

Inverse Kinematics: The Challenges

CORE

A target pose may have zero, finitely many, or infinitely many joint solutions

No solution

The requested position or orientation lies outside the robot's feasible task space.

Finite solutions

A regular six-axis robot may return 2, 4, 6 or 8 branches before limits and collisions filter them.

Infinite family

At some singularities, a continuum of joint states produces the same pose. A wrist singularity couples q_4 and q_6.

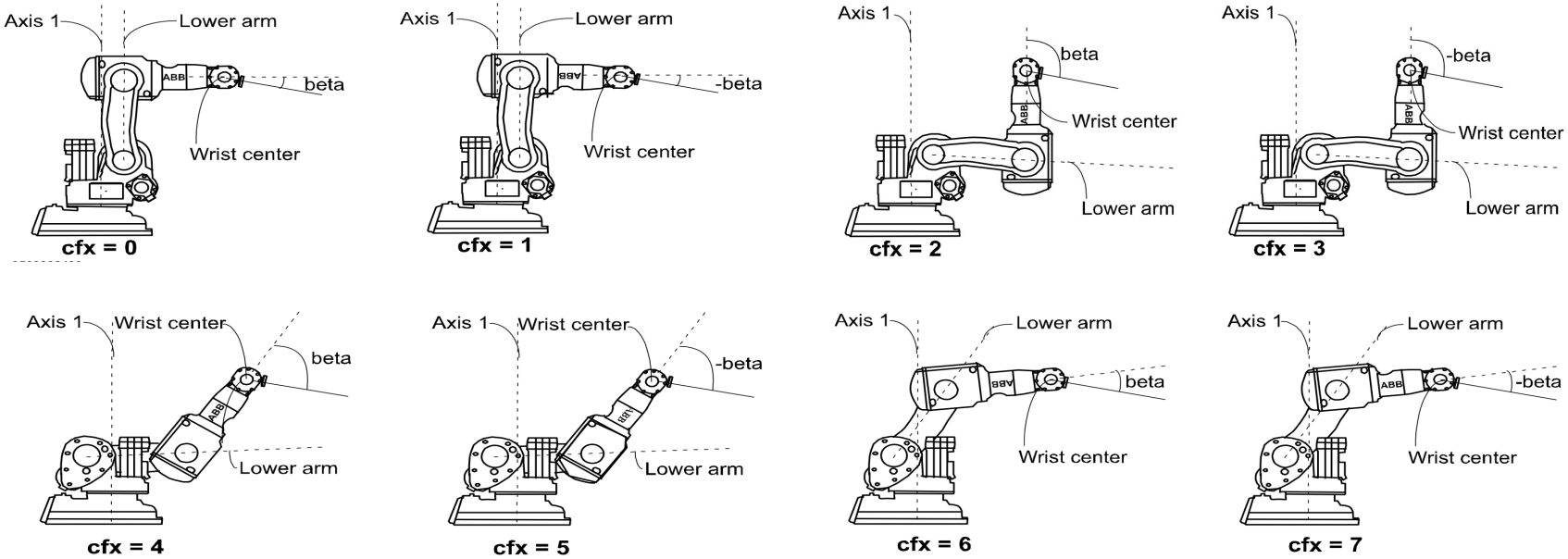
Singularity: the Jacobian loses rank. Cartesian motion or force may demand very large joint rates, and the IK branch can become ambiguous.

IK must return a feasible branch and explicitly flag singular or near-singular cases.

Robot Configurations

CORE

Shoulder, elbow and wrist choices — up to eight ways to reach one pose



A configuration code identifies the shoulder, elbow and wrist branch.

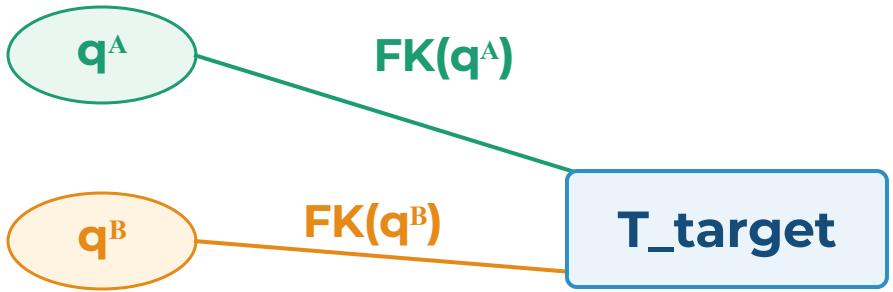
The next slide explains why pose alone does not select a unique solution.

Pose and Configuration

CORE

An execution target needs a pose and a configuration

execution target = (target pose, configuration)



$q^A \neq q^B$, yet

$FK(q^A) = FK(q^B) = T_target$

The pose identifies where the tool goes. The configuration identifies which robot branch executes the move.

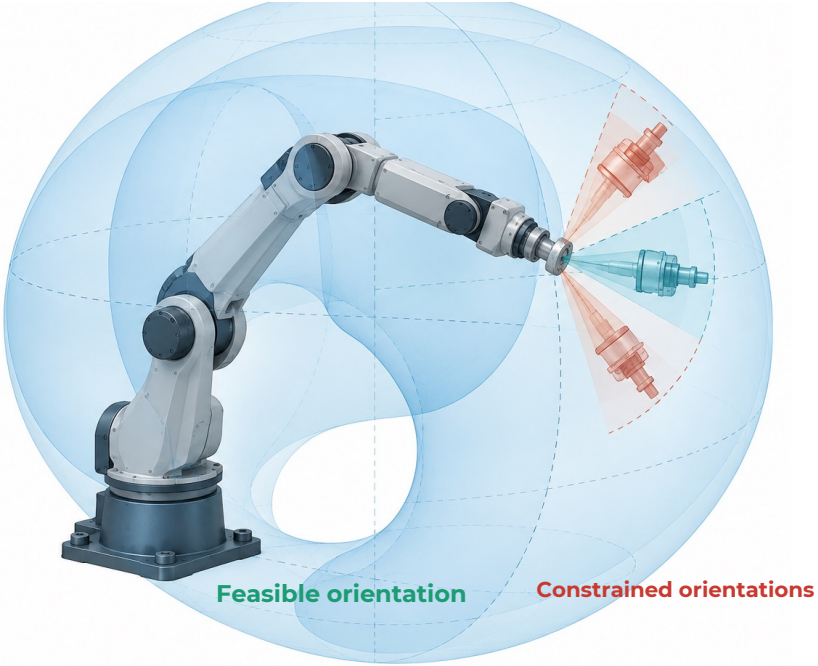
Position and Pose Reachability

CORE

Task space is six-dimensional

pose = (x, y, z, roll, pitch, yaw)

The translucent envelope shows reachable positions only. At the same TCP point, one tool orientation may be feasible while another violates joint limits, wrist geometry or collision clearance.



A position workspace is only a 3D projection of the feasible 6D pose set.

PART 2

Analytic branches and reachability

Geometry explains branches, boundaries and joint-limit filtering

1 Planar inverse-kinematics example
2 Workspace and solution boundaries
3 Branch count, limits and controller configuration

Example: Planar 2R Robot

Two revolute joints, fixed link lengths, a position target

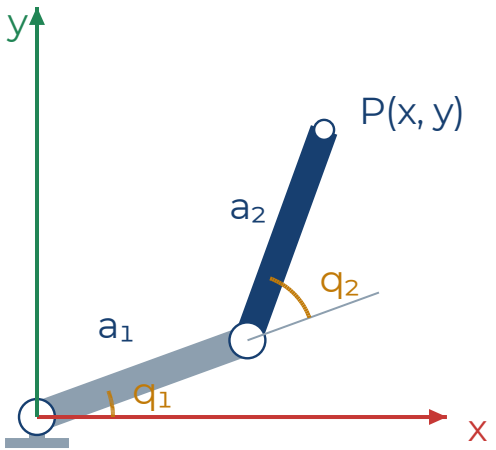
$$x = a_1 cos q_1 + a_2 cos(q_1 + q_2)$$

$$y = a_1 sin q_1 + a_2 sin(q_1 + q_2)$$

Forward kinematics is two equations. Inverse kinematics solves the same two for q_1 and q_2.

Angles are counter-clockwise positive; q_2 is measured from link 1.

Solve the position first. The tool angle is then $q_1 + q_2$.



Example: Elbow Angle

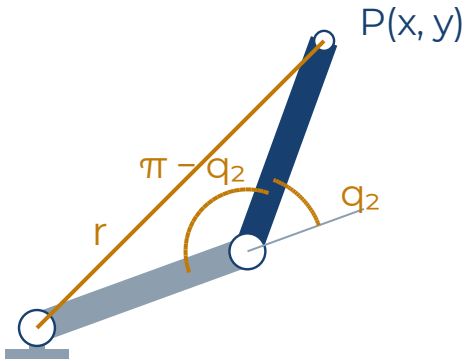
The distance to the target fixes the angle between the links

$$c_2 = \frac{x^2 + y^2 - {a_1}^2 - {a_2}^2}{2a_1 a_2}$$

$$q_2 = atan2\left(\pm\sqrt{1 - {c_2}^2}, c_2\right)$$

$|c_2| > 1$: the target is out of reach.

The $\pm$ sign gives the two elbow branches.



Recover the angle with atan2, not acos: acos returns only one branch.

Example: Shoulder Angle

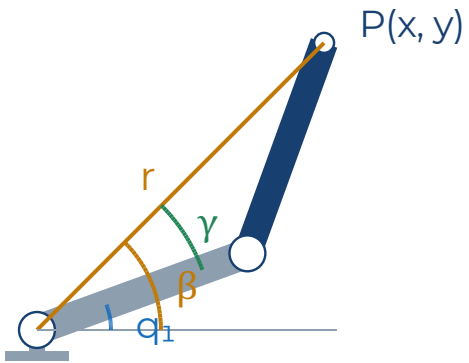
Subtract the triangle angle from the direction to the target

$$q_1 = atan2(y, x) - atan2(a_2 sinq_2, a_1 + a_2 cosq_2)$$

Evaluate q_1 once for each q_2 branch — the two are paired.

atan2(y, x) keeps the quadrant that atan(y/x) loses.

Two branches in, two complete solutions out.

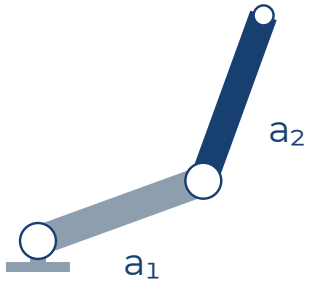


Example: Numerical Solution and FK Check

EXAMPLE

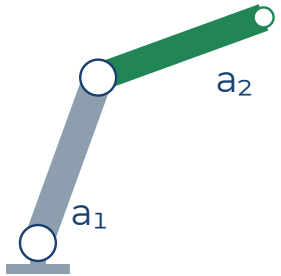
$a_1 = a_2 = 1$ m, $P = (1.281713, 1.281713)$ m (rounded)

P = (1.2817, 1.2817)



A: (20°, 50°)

P = (1.2817, 1.2817)



B: (70°, -50°)

$c_2 \approx 0.6428$, so $q_2 = \pm 50°$. A: (20°, 50°), B: (70°, −50°).

FK: x = cos20° + cos70° ≈ 1.281713 m; y = sin20° + sin70° ≈ 1.281713 m.

Both branches reach P. Their tool orientations are 70° and 20°.

Workspace and Solution Boundaries

CORE

Six-axis robot with tool, joint limits and self-collision considered

Outer reach

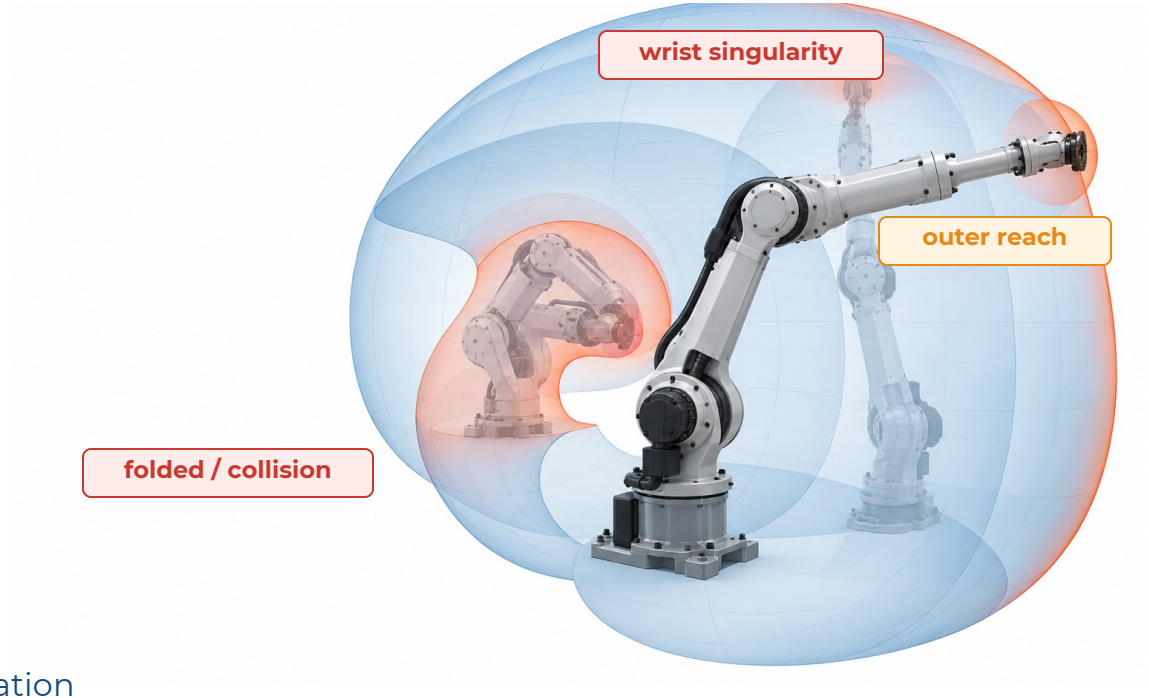
Shoulder and elbow approach full extension. IK branches merge and manipulability falls.

Inner excluded regions

Folded postures, link self-collision, the base, fixture and tool clearance remove space from the ideal reach envelope.

Orientation boundary

A TCP position may lie inside the envelope while the required orientation places the wrist at or near a singularity.



Joint Limits Filter IK Branches

CORE

A classical 6R robot with a spherical wrist has at most eight analytic branches

Solution	q_1	q_2	Within −10° … 100° ?
A	20°	50°	yes
B	70°	−50°	no — q_2 is below the lower limit

Three inverse-trigonometric steps can each create two branches

Shoulder
left / right
2 choices

Elbow
up / down
2 choices

Wrist
not flipped / flipped
2 choices

2 × 2 × 2 = 8
maximum candidates

Joint limits, self-collision, singularity and mechanical restrictions reject candidates. A specific target may leave 4, 2, 1 or 0 feasible solutions.

The eight-solution result is a structural maximum, not a guarantee for every robot or every target.

Configuration Choice in a Robot Program

CORE

General planning principle

Choose one feasible IK branch and maintain continuity through neighbouring targets. Validate intermediate poses for joint limits, collision and singularity proximity.

Controller-language implementation

ABB RAPID uses MoveL and MoveJ motion instructions, robtarget configuration data, and configuration checking. Other robot brands may use different commands and branch encodings.

Path feasibility still matters

Two valid endpoint solutions do not prove that the commanded path between them is valid. The controller must use the intended motion type and configuration behaviour.

ABB-specific commands return in the ABB programming lecture. Here the key decision is the intended branch and path behaviour.

PART 3

Numerical IK and singularities

Iterative solution methods and their physical limitations

1. Analytic and numerical IK
2. Initial guesses and convergence
3. Singularity demonstrations and method selection

Analytical and Numerical IK

CORE

Two ways to solve the same kinematic constraints

Analytical (closed form)

Algebra derived once from the geometry.

Fast, exact, returns every branch.

Depends on robot geometry; **a spherical wrist is one case.**

Numerical (iterative)

Repeatedly corrects a joint estimate until the pose error is small.

Works for any structure, including redundant arms.

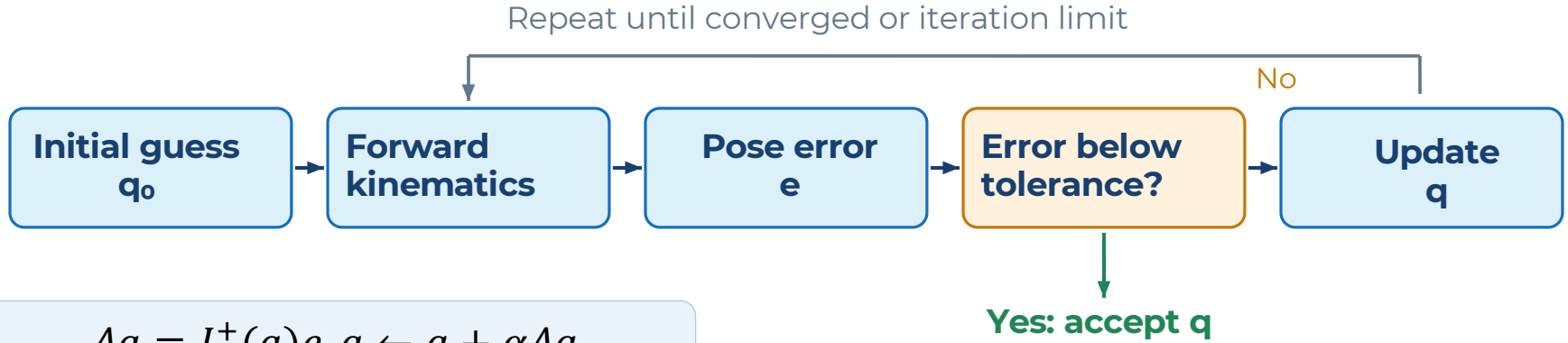
Returns one solution, dependent on the starting guess.

Use the analytic solution when the robot has one. Fall back to the numerical solver when it does not.

How Numerical IK Works

CORE

The iterative correction has a direct Corke RTB10 implementation



$$\Delta q = J^{+}(q)e, q \leftarrow q + \alpha \Delta q$$

The Jacobian maps pose error into a joint correction. Near a singularity, damping controls unstable updates. Position and orientation errors need suitable weighting.

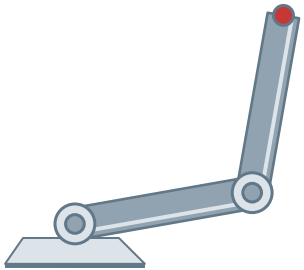
```
% rob and T: Puma example on slide 30
q0 = [2.7 2 -0.1 0 1.3 2.7];
q = rob.ikine(T,'q0',q0, ...
              'tol',1e-10,'ilimit',500);
assert(~isempty(q) && all(isfinite(q)));
Tcheck = rob.fkine(q);
```

Use the model and target on slide 30. A failed iteration may return empty q: check it before FK, then check pose error and travel limits.

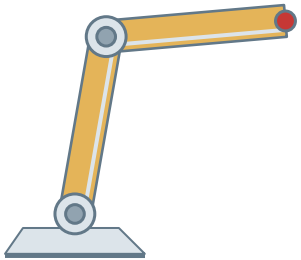
Why the Initial Guess Matters

CORE

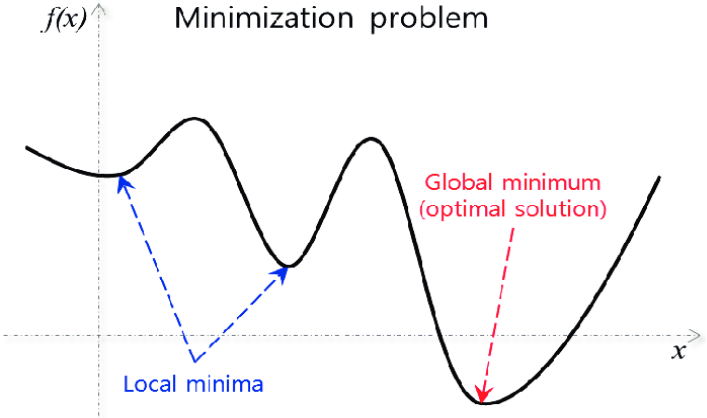
The same target and different q0 values can converge to different configurations



Seed near A



Seed near B




```
% rob and T as on slide 30
q_seed_A = rob.ikine6s(T,'lun');
q_seed_B = rob.ikine6s(T,'rdf');
qA = rob.ikine(T,'q0',q_seed_A);
qB = rob.ikine(T,'q0',q_seed_B);
```

Different seeds may find different IK branches. Both can reproduce T, but an algebraic solution can still violate travel limits. Verify each result.

A failed numerical iteration does not prove the target is unreachable. Change the seed, then check limits and pose error.

Singularity: Physical Meaning

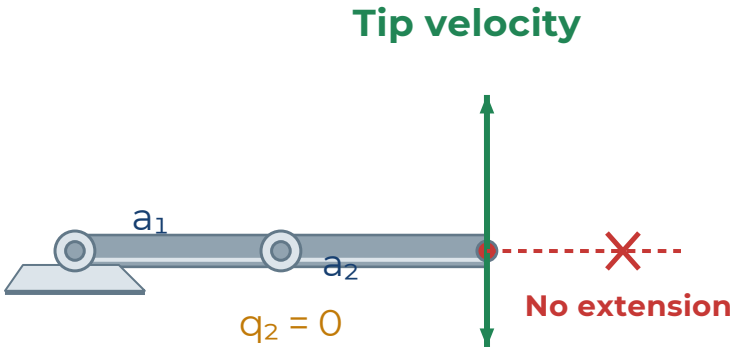
CORE

A direction of tool motion that is no longer available

$$detJ(q) = 0$$

With the arm straight, the tip cannot be extended further to first order.

Near a singularity a modest tool speed demands very large joint speeds.



Singular postures are a property of the robot, not of the solver — plan around them.

Singularity Demonstration

CORE

External video demonstration

▶ **Open singularity demonstration**

Six-axis robot singularity motion · 1 min 29 s

The video is stored in the course videos folder and is not embedded in this PowerPoint.

PART 4

Spherical wrist and toolbox IK

A useful special structure and the MATLAB tools that use it

1. Wrist decoupling and orientation solution
2. Analytic and numerical toolbox commands
3. Project targets, checks and common errors

Spherical Wrist: The IK Simplification

OPTIONAL

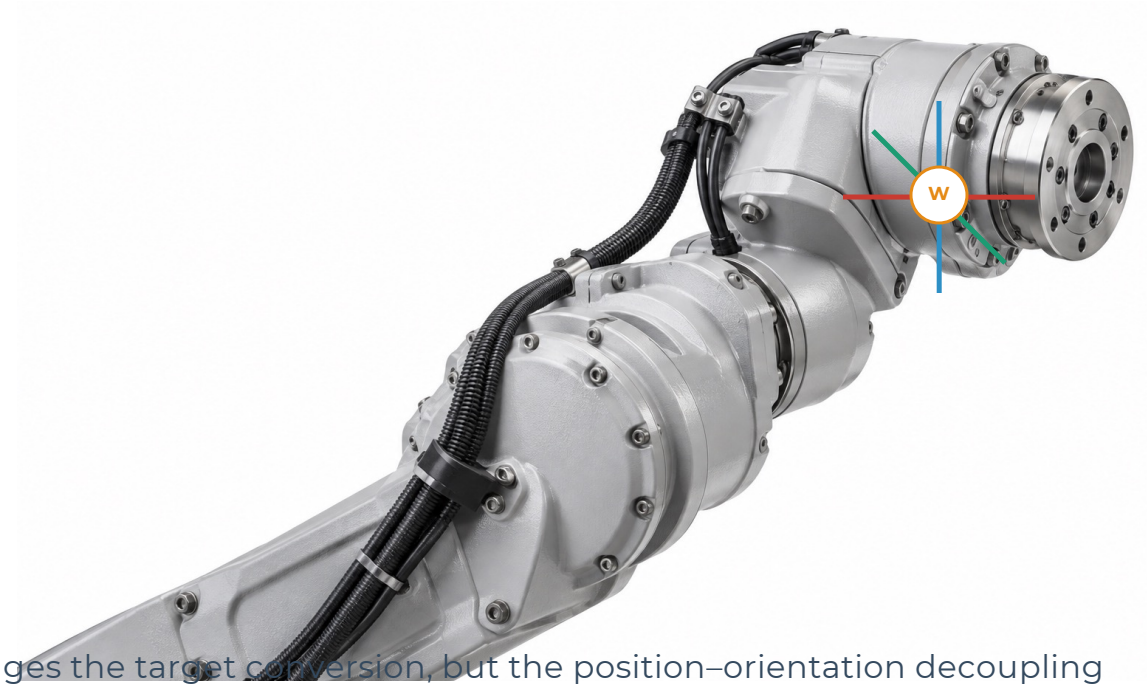
Joint axes 4, 5 and 6 intersect at one wrist centre W

Solve through the wrist centre

1 Convert the TCP target to the wrist-centre position W.

2 Solve q_1, q_2, q_3 from W.

3 Solve q_4, q_5, q_6 from the remaining orientation.



A fixed flange or tool offset changes the target conversion, but the position–orientation decoupling remains.

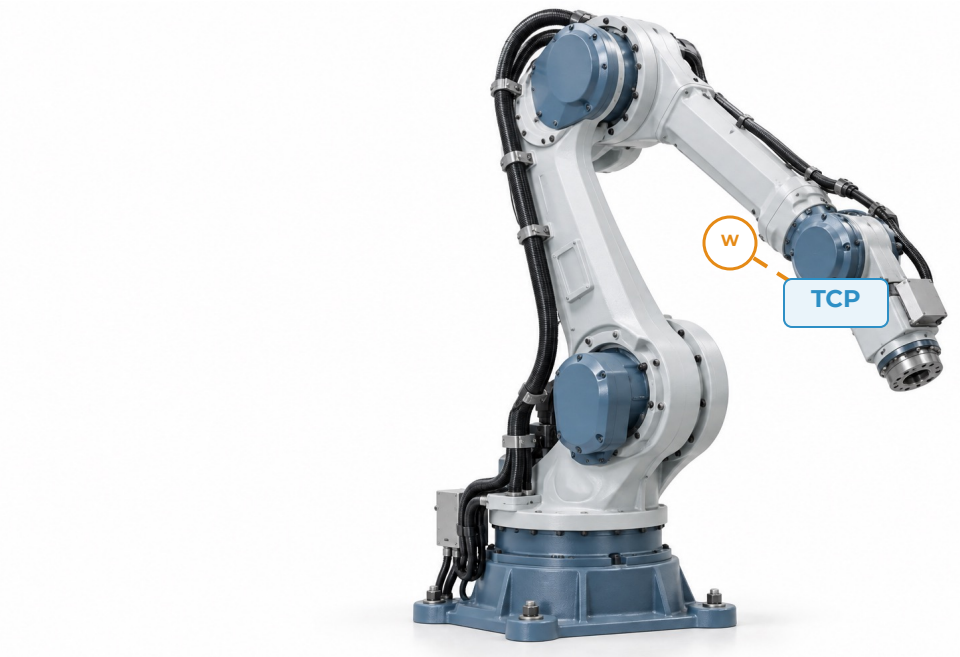
Spherical and Offset Wrists

OPTIONAL

Wrist geometry determines the IK method

Spherical wrist

Axes 4–6 intersect at W.
Compute W from the target pose, then solve position and orientation separately.



Offset wrist

The axes do not share one point.
Internal offsets couple position and orientation. Use a robot-specific analytic solver or numerical IK.

Tooldata represents a fixed transform after the flange; it cannot remove offsets between the wrist axes.

Which Toolbox IK Method

CORE

Peter Corke RTB10: ikine6s and ikine

Method	Kind	Use it when
ikine6s	analytic	Supported 6-axis geometry with a spherical wrist and standard DH
ikine	numerical	General numerical IK; choose a seed and verify the result and limits

Use ikine6s only when the robot model matches its supported spherical-wrist structure.

ikine6s — the Analytic Solution

CORE

Corke RTB10 analytic IK for supported spherical-wrist models

```
mdl_puma560             % defines p560
T = transl(.6,-.1,.3) * ...
    rpy2tr(0,180,0,'deg');
q  = p560.ikine6s(T,'lun');
qr = p560.ikine6s(T,'ru');
assert(~isempty(q) && all(isfinite(q)));
poseError = norm(p560.fkine(q).T-T,'fro');
```

Requires supported standard-DH geometry and a spherical wrist; unsupported models can raise an error.

Select the branch with single-quoted character codes.

An unreachable target can warn and return NaN. Check finite values, FK error and limits.

Use ikine6s only for compatible geometry; otherwise solve IK for the real robot.

The Configuration String

CORE

Three letters select which of the eight branches is returned

Choice	Letters	Default
Shoulder	l left r right	l
Elbow	u up d down	u
Wrist	n not flipped f flipped (180°)	n

```
% p560 and T from slide 28
qA = p560.ikine6s(T,'lun');
qB = p560.ikine6s(T,'rdf');
```

Omitted letters take the default.

Use 'lun' and 'rdf', not string objects "lun" and "rdf", with this RTB10 API.

An unavailable branch can warn and return NaN.

Hold the branch along a path, then verify poses and limits.

These three letters are the same three binary choices as the controller configuration code.

ikine — the Numerical Solution

CORE

Corke RTB10 numerical IK for other robot models

```
mdl_puma560; rob = p560;
T = transl(.6,-.1,.3)*rpy2tr(0,180,0,'deg');
q0 = [2.7 2 -0.1 0 1.3 2.7];
q = rob.ikine(T,'q0',q0, ...
             'tol',1e-10,'ilimit',500);
assert(~isempty(q) && all(isfinite(q)));
Tcheck = rob.fkine(q);
poseError = norm(Tcheck.T-T,'fro');
qlim = rob.qlim;
withinLimits = all(q >= qlim(:,1).' & ...
                   q <= qlim(:,2).');
```

Option	Meaning
q0	starting estimate (default zeros)
mask	which Cartesian DoF to enforce
tol	convergence tolerance (1e-10)
ilimit	iterations per try (500)

ikine does not enforce joint limits. Accept a solution only after convergence, FK residual and travel-limit checks.

Spherical Wrist and Solver Choice

CORE

Geometry, DH convention and the limits of ikine6s

Physical wrist geometry

Spherical wrist
Axes 4, 5 and 6 intersect.
The arm geometry must also match a case supported by ikine6s.

A genuinely offset wrist needs a matching analytical solver or numerical IK.
An equivalent DH redefinition must preserve FK and the joint mapping.

DH and solver compatibility

A convention mismatch
isspherical checks a specific DH pattern.
A false result may reflect frame choices,
even when the physical wrist is spherical.

Recommendation: check the physical axes and DH convention before choosing a solver.
Keep real dimensions, then verify the pose, joint limits and singularity status.

Two Different Wrist Models

EXAMPLE

Equivalent target conversion versus a genuinely offset wrist

IRB2600ID: spherical wrist

Keep the original DH geometry.
$\alpha_4 = \alpha_5 = +90°$, joint-5 offset = 180°.
Remove base/tool transforms, then subtract $d_6 = 200$ mm along flange z to obtain the wrist-centre target.

UR10e: offset wrist

Keep the real wrist offsets (mm):
$d_4 = 174.15$, $d_5 = 119.85$, $d_6 = 116.55$.
BJTUcode uses a dedicated UR solver. Setting d_5 to zero would change the robot geometry.

Puma 560: supported Toolbox model

```
mdl_puma560; T=p560.fkine([0 .5 -.5 0 .5 0]); q=p560.ikine6s(T,'lun');
```

Recommendation: if your own model cannot be solved, use Puma 560 with ikine6s for teaching IK only. It does not validate your project robot; use metres.

Project: Critical Assembly Poses

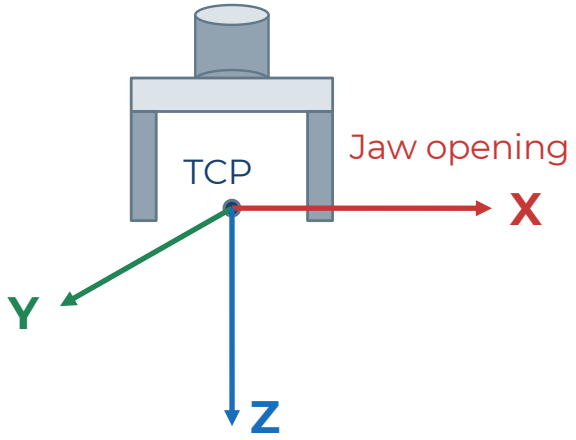
CORE

Pick-up and insertion need a consistent TCP and approach direction

Define and justify your tool frame: align one axis with the approach; choose the other axes for your end-effector.

Solve IK at the pick pose, the insertion pose and one approach pose above each.

Record the configuration used, and keep it the same across the sequence.



The drawn x-jaw frame is an example; transform L2 targets to your chosen project tool frame.


UNIVERSITY
OF WOLLONGONG
AUSTRALIA
Stand
for purpose
THANK YOU!